

Master Ingénierie Electrique et Systèmes Embarqués IESE

Cours

Langage VHDL

Mouhcine RAZI

mouhcine.razi@usmba.ac.ma

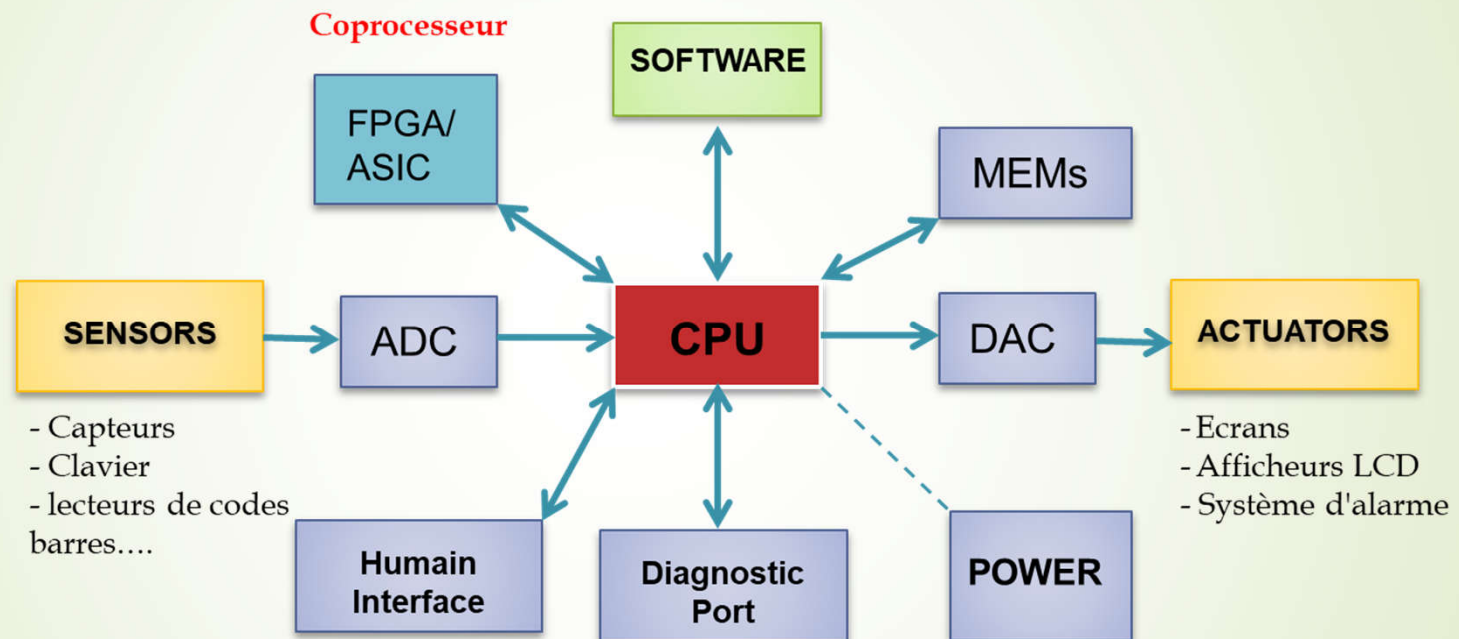
*Année universitaire
2025/2026*



Plan

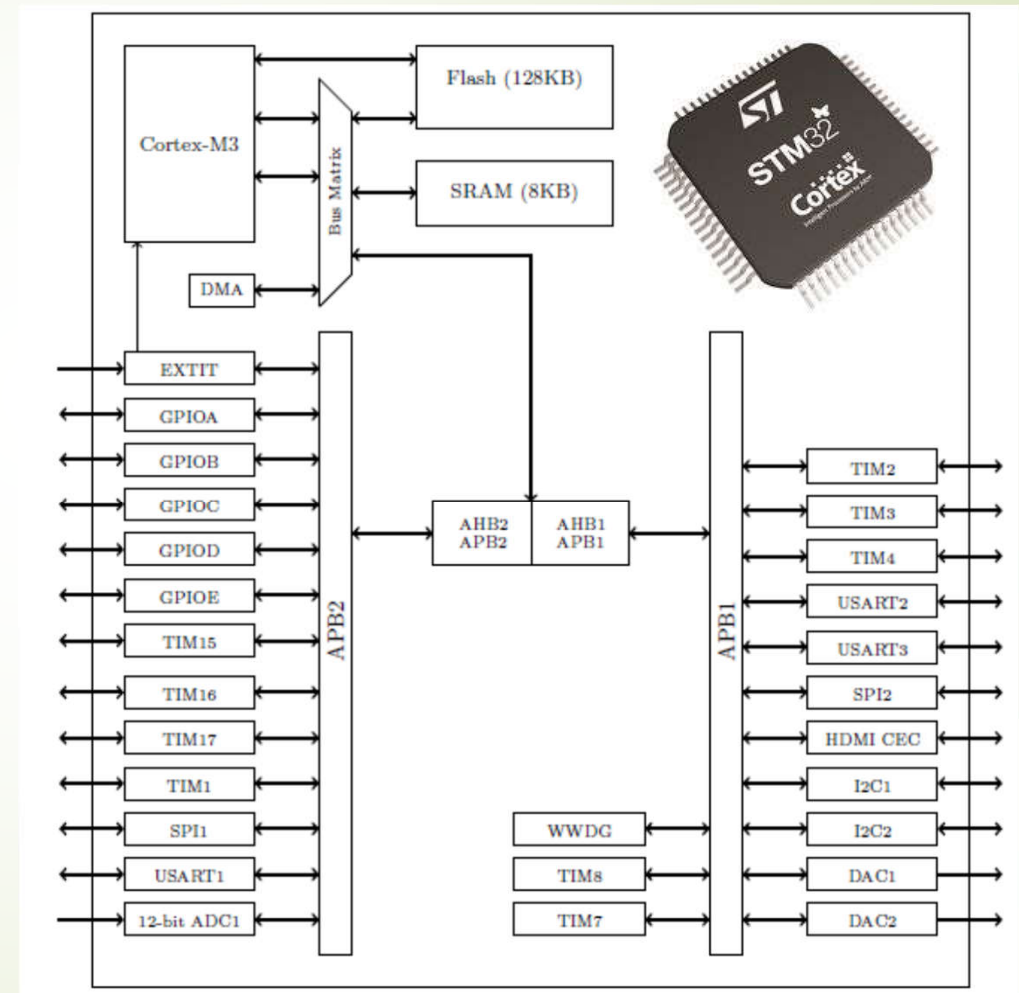
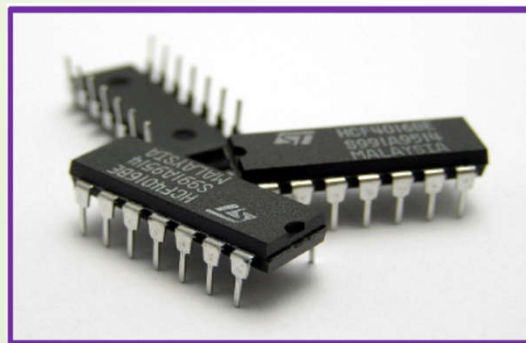
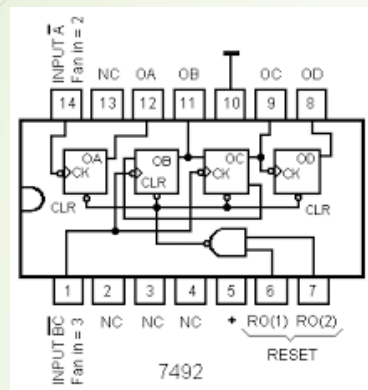
- Introduction et historique du langage VHDL
- Définition du langage VHDL
- Flot de conception
- Modélisation et synthèse
- Unités de conception
- Généralités sur le langage
- VHDL concurrent et séquentiel
- Simulation par testbench

Introduction



Architecture générale d'un SE typique

Introduction



Historique

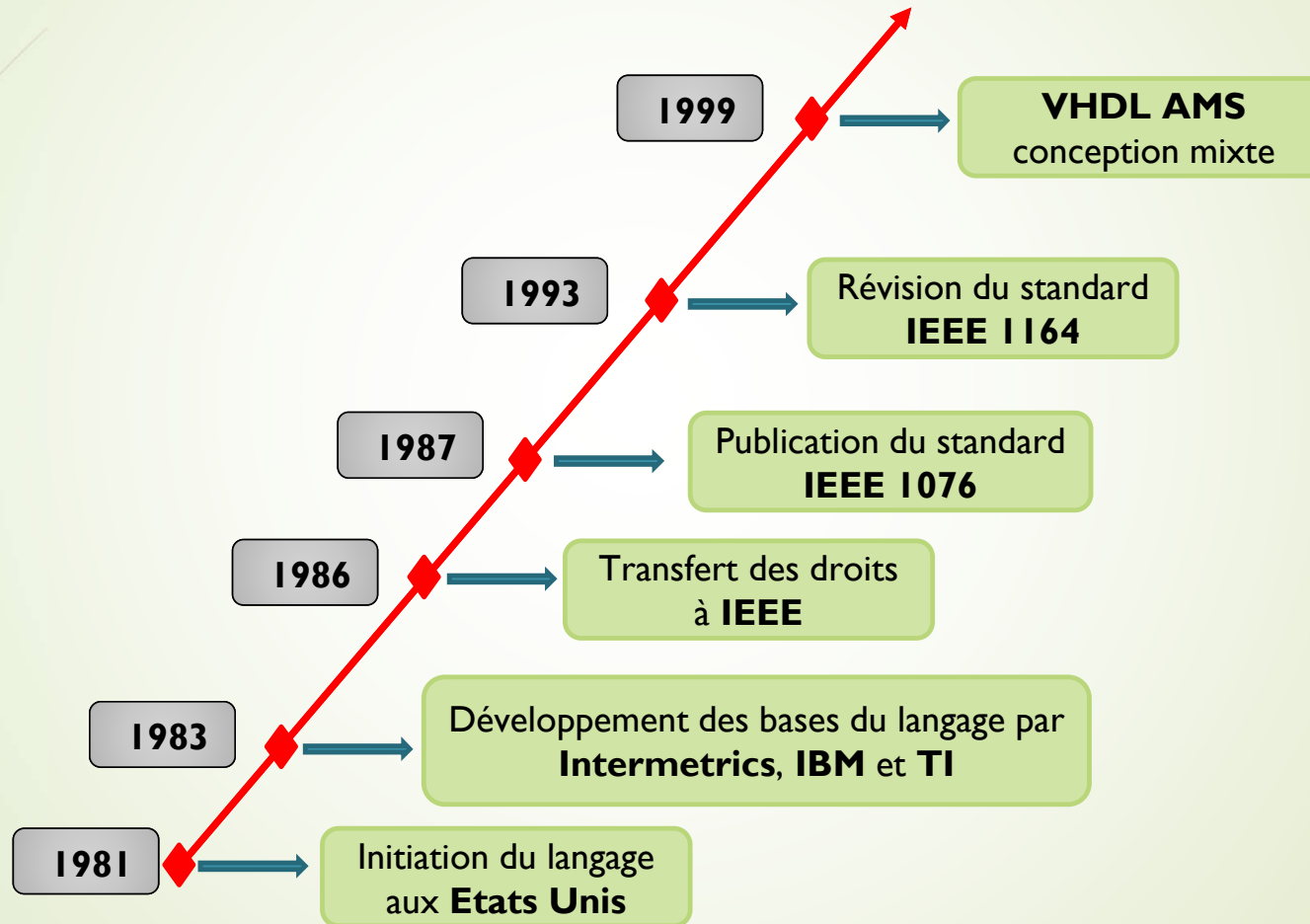
Langages de description de matériel

Les premiers circuits intégrés numériques étaient décrits par des logigrammes. On les dessinait dans des logiciels de saisie de schémas. La complexité des circuits croissant, les fabricants ont développé des outils plus performants et plus souples à utiliser que la saisie de schéma.

La création du langage **VHDL** vient d'une demande du gouvernement américain, qui a amorcé ce projet dans les années **1980**. Les États-Unis voulaient un langage de description de **matériel de haut niveau** pour décrire les circuits intégrés numériques à architectures spécifiques (**ASIC**).

Le **VHDL** a été normalisé en 1987 (norme IEEE 1076), sous l'appellation courante VHDL-87. En 1993, une version beaucoup plus aboutie de la norme IEEE 1076 est adoptée, sous l'appellation VHDL-93 (norme **IEEE 1164**). Depuis, il est régulièrement amendé et enrichi au travers de nouvelles révisions : VHDL-1997, VHDL-2002, VHDL-2008...

Historique



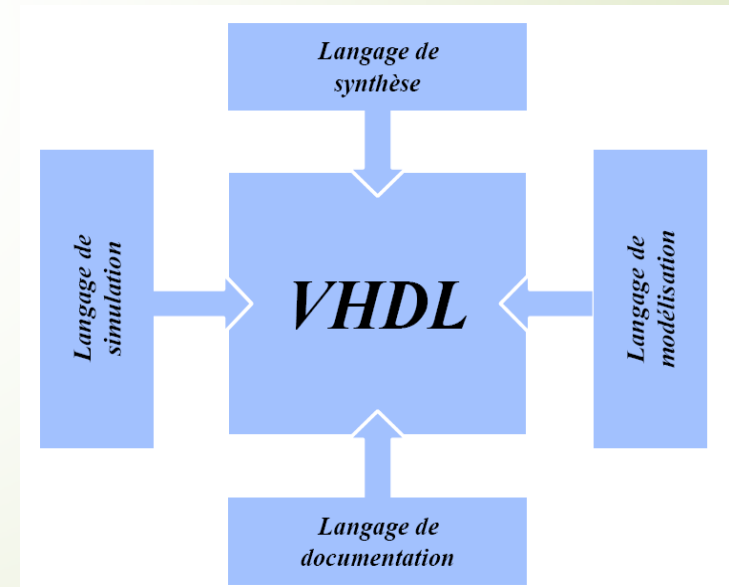
Définition

Monde des Processeurs (μP , μC , DSP)	Monde des circuits HW ($ASIC$, PLD)
✓ Architecture fixée par le constructeur (CPU, RAM, ROM, EPROM, Périphériques,...) ✓ Exécution séquentielle des instructions ✓ Fréquence du fonctionnement limitée par le mode séquentiel	✓ Architecture conçue par l'utilisateur ✓ Les modules électroniques du circuit conçu fonctionnent en mode parallèle et/ou séquentiel ✓ Fréquence de fonctionnement élevée (mode parallèle)

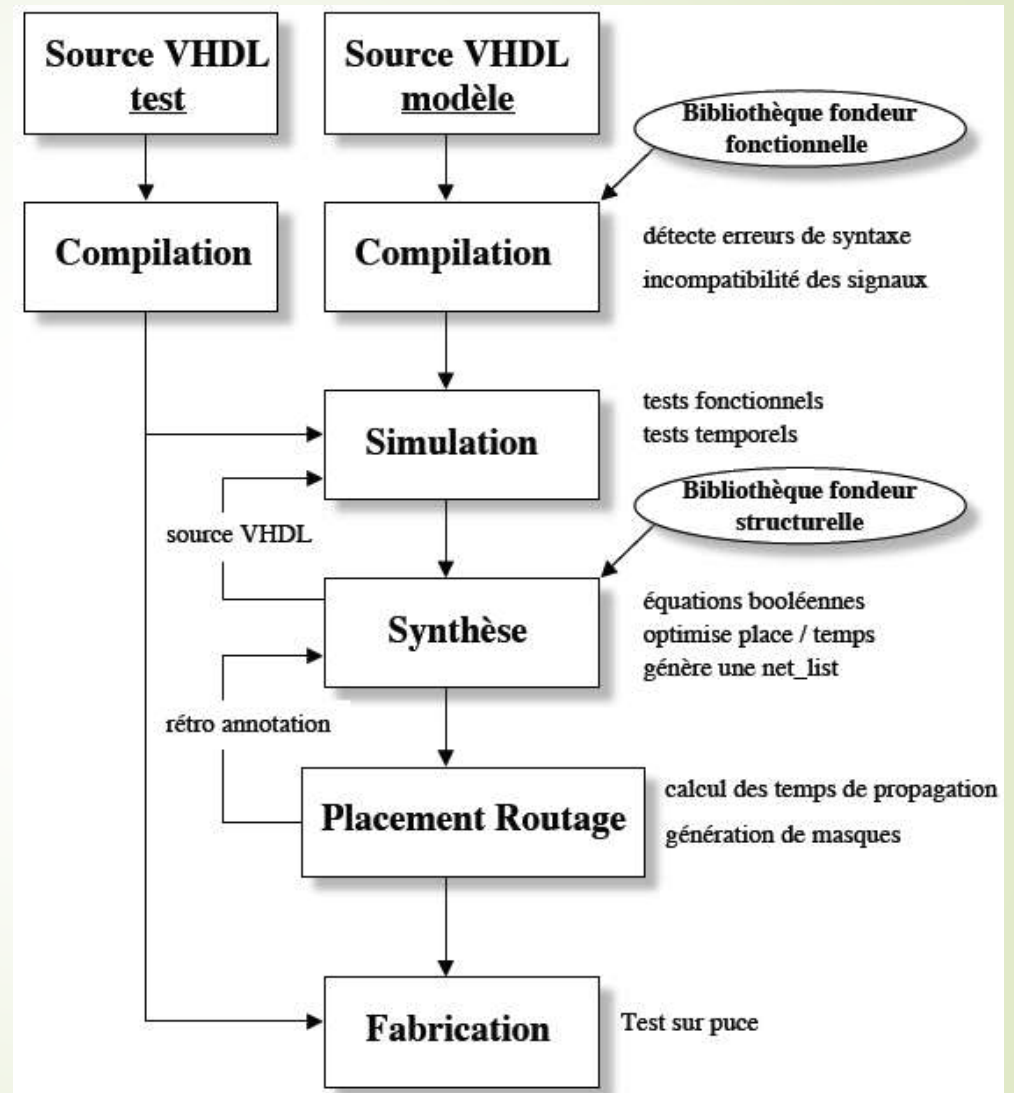
Définition

Le **VHDL** est un langage fonctionnel de haut niveau qui ne vise pas une exécution, il est utilisé pour :

- La modélisation des systèmes électroniques complexes.
- La simulation du comportement des modèles.
- La synthèse
- La documentation.



Flot de conception



Utilisation du VHDL

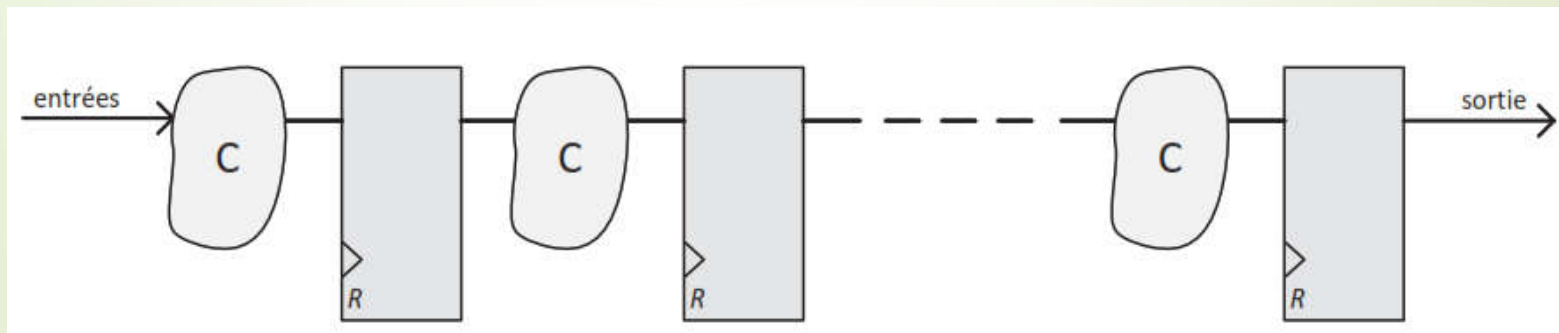
VHDL pour la modélisation

Avant de créer matériellement un circuit, il faut souvent valider le principe et la faisabilité du circuit. C'est pourquoi il faut créer un **modèle** en VHDL et le valider. Dans ce cadre, on utilise des instructions qui **ne sont pas nécessairement synthétisables**, comme la division, les fonctions trigonométriques ou autres car l'objectif est de valider le concept du circuit, non de le réaliser.

Utilisation du VHDL

VHDL pour la réalisation du circuit

On parle dans ce cas de VHDL **synthétisable**. En effet, on ne doit utiliser que des instructions qui soient compréhensibles par l'outil de synthèse. En effet, c'est lui qui permet de transformer le texte VHDL en un ensemble de portes logiques et de registres. Pour pouvoir être synthétisable correctement, la description doit être RTL (Register Transfer Level). On rappelle que la description RTL est construite comme la succession d'alternance de blocs de logique combinatoire et de registres.



Description RTL

Limitations du VHDL

Création des modèles de simulations:



NORME IEEE VHDL

La totalité de la norme peut être utilisée pour réaliser des modèles de simulations.

- ✓ Tout le langage:
Logique + Temporel
- Exemple:
Modélisation des vecteurs de test

Création des circuits intégrés:



NORME IEEE VHDL

Seule une partie de la norme peut être utilisée pour réaliser des circuits.

- ✓ Langage simplifié (pas de retards).
- ✓ Le style d'écriture anticipe une primitive du circuit.
- ✓ Evaluation des performances
- ✓ La synthèse demande une bonne connaissance du circuit et de la technologie.

Modélisation

- Un **modèle** peut être:
 - Comportemental,
 - Structurel ou
 - Data-flow
- La faisabilité **matérielle** n'est pas nécessairement prise en compte à ce stade.
- Le **partitionnement** en plusieurs sous-ensembles permet de subdiviser un modèle complexe en un certain nombre de blocs prêts à être développés séparément.

Simulation

- C'est un des points fort des HDL. Ils permettent de simuler un circuit en utilisant le même langage.
- Pour simuler un circuit, on doit construire des chronogrammes en faisant varier des signaux dans le temps.
- Un environnement de simulation complet comprend:
 - ✓ Le modèle du circuit à simuler.
 - ✓ Le générateur de stimuli.
 - ✓ Un vérificateur automatique de résultats

Synthèse

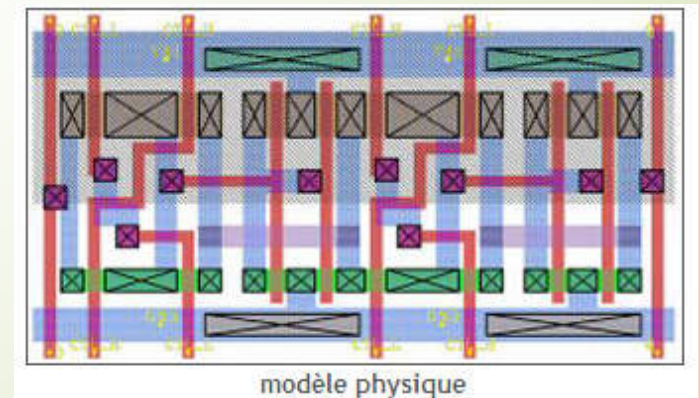
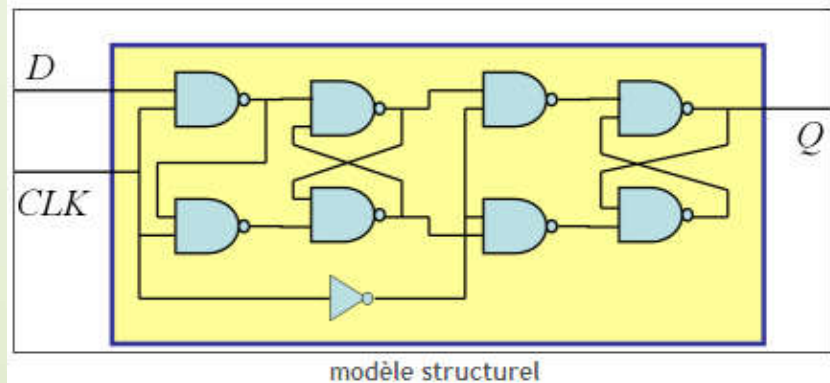
Le passage d'une vue (modèle) à l'autre s'appelle **synthèse**.

✓ Synthèse logique:

C'est le passage d'une vue comportementale à une vue structurelle. Elle est souvent *automatique*, traitée par des logiciels spécialisés et complexes.

✓ Synthèse physique:

Passage d'une vue structurelle à une vue physique. On part d'une liste de portes et de leurs interconnexions (*netlist*) pour aboutir à une description physique du circuit en termes de masques. Fonction de **placement et routage** de cellules (transistors).



Synthèse

- Les langages fonctionnels permettent de concevoir des circuits qui seront véritablement fabriqués.
- Prendre en considération **les limites des outils logiciels**:
 - ✓ Traduction du code en portes logiques (synthétiseurs)
 - ✓ Description structurelle en dessin au micron (placeurs- routeurs)
 - ✓ Production d'une description structurelle la plus économique possible (surface de silicium la plus petite possible).

Utilisation du VHDL

A ce jour on utilise le langage VHDL pour :

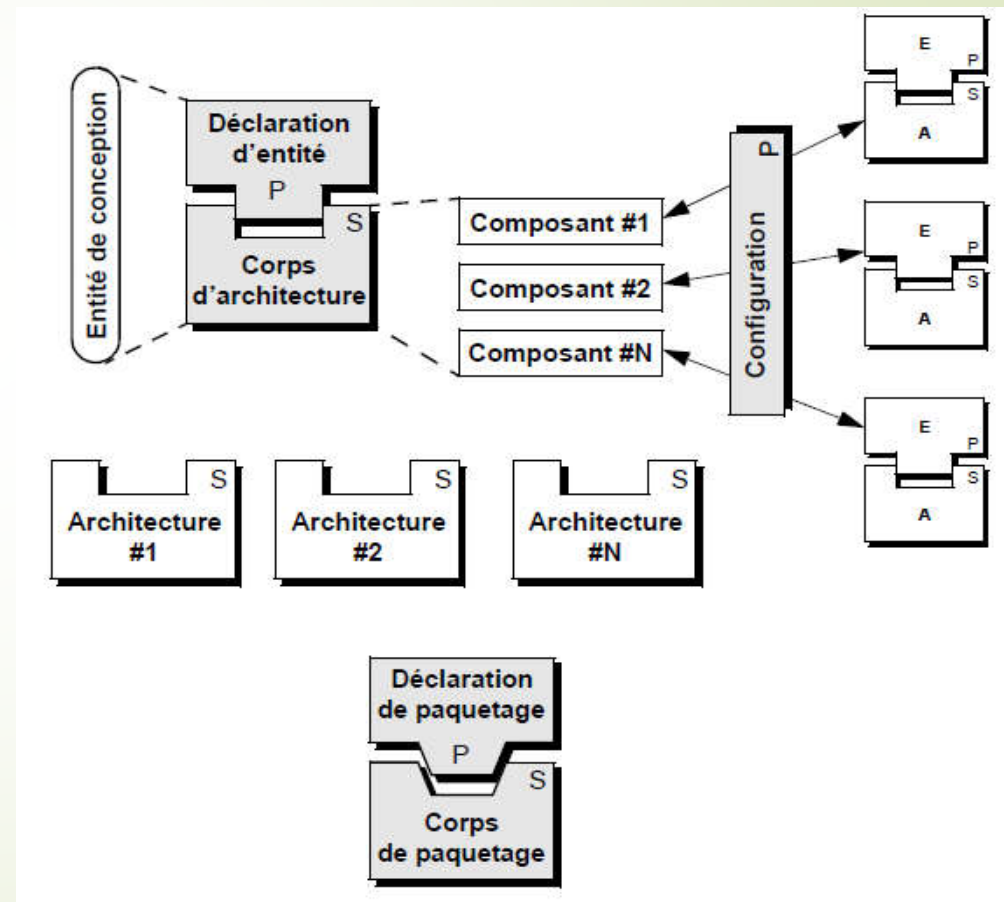
- ✓ Concevoir des circuits spécifiques (ASICs)
- ✓ Configurer des circuits programmables comme les PLDs, CPLDs et FPGAs.

Unités de conception en VHDL

L'unité de conception (**design unit**) est le plus petit module compilable séparément.

VHDL offre cinq types d'unités de conception :

- La déclaration d'entité;
- Le corps d'architecture;
- La déclaration de configuration;
- La déclaration de paquetage;
- Le corps de paquetage (package body).



Généralités sur le langage

Entité de conception

Un code VHDL complet comporte 3 parties :

- La liste des **bibliothèques** utilisées par le code.
- La description de l'**entité** du circuit.
- La description de l'**architecture** du circuit.

Les bibliothèques VHDL

Toute description VHDL utilisée pour la synthèse a besoin de **bibliothèques**. **L'IEEE** (Institut of Electrical and Electronics Engineers) les a normalisées et plus particulièrement la bibliothèque **IEEE 1164**. Elles contiennent les définitions des types de signaux électroniques, des fonctions et sous programmes permettant de réaliser des opérations arithmétiques et logiques,...

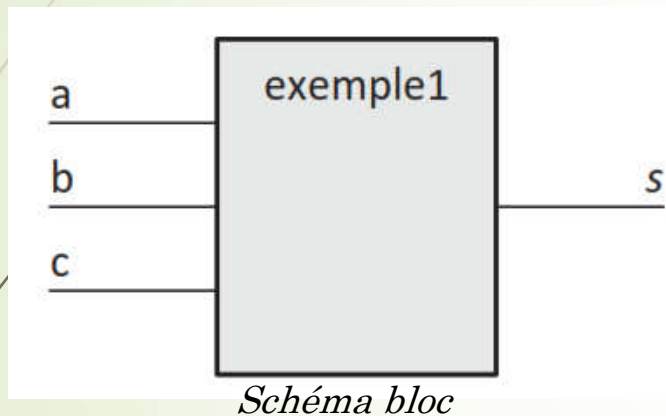
Les bibliothèques VHDL

Pour réaliser les opérations arithmétiques on peut utiliser les paquetages dans la bibliothèque IEEE :

- ✓ `Use ieee.std_logic_arith.all;`
- ✓ `Use ieee.std_logic_unsigned.all;`
- ✓ `Use ieee.std_logic_signed.all;`
- ✓ `Use ieee.numeric_std.all;`

Description de l'entité du circuit

Cette description VHDL contient la liste de toutes les entrées sorties du circuit. Cela correspond au schéma bloc du circuit.



```
entity exemple1 is
port
    (a      : in  std_logic;
     b, c   : in  std_logic;
     s      : out std_logic);
end entity;
```

Code de l'entité

Notons que :

- Ce circuit possède 3 entrées et 1 sortie.
- La dernière sortie déclarée ne se termine pas par ";".
- L'ordre dans lequel sont écrites les entrées/sorties est sans importance.
- Le type "std_logic" sera expliqué par la suite.

Description de l'entité du circuit

En plus de la liste des entrées sorties, on peut avoir une liste de paramètres, que l'on appelle des paramètres **génériques**. Cela permet de créer un circuit de manière globale. On fixera la valeur des paramètres génériques pour la simulation ou lors de la réalisation en fonction des besoins.

Par exemple :

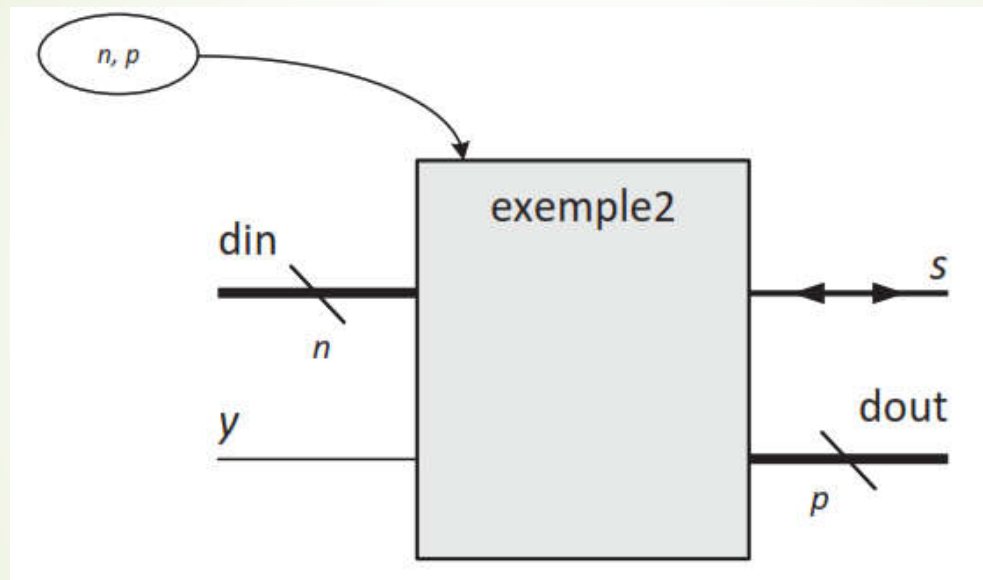
```
entity exemple2 is generic (n: natural:= 8; p: natural:= 4);
port
    (din      : in  std_logic_vector(n-1 downto 0);
      s        : inout std_logic;
      dout     : out std_logic_vector(p-1 downto 0);
      y        : in  std_logic);
end entity;
```

On a 2 paramètres génériques n et p . Leurs valeurs par défaut sont respectivement 8 et 4.

Dans la liste des entrées sorties, on trouve : Un bus en entrée "din", sur n bits, un bus en sortie "dout" sur p bits, une entrée simple "y" et une entrée sortie bidirectionnelle "s".

Description de l'entité du circuit

Cette entité correspond au schéma bloc suivant:



Grace à ce paramètre générique, on peut décrire un circuit de manière globale puis l'adapter à nos besoins sans avoir à retoucher le code VHDL.

Description de l'architecture

C'est dans cette partie du code que l'on décrit le fonctionnement du circuit. L'architecture doit avoir un nom et elle dépend des entrées sorties du circuit. C'est pourquoi elle est forcément liée à une entité. Cela se traduit dans la syntaxe de la déclaration :

```
■ architecture X1 of exemple1 is
```

Notons qu'ici le nom de l'**architecture** est "*X1*". Elle est associée à une **entité** dont le nom est "*exemple1*".

Description de l'architecture

Après la déclaration se trouve une zone où est listé tout le matériel nécessaire à la description de l'architecture. On y trouve la liste de toutes les liaisons physiques permanentes internes à l'architecture. On appelle ces connections "**signal**" en VHDL.

Une fois cette liste terminée, on décrit l'architecture entre les instructions "*begin*" et "*end architecture*". Par exemple :

```
architecture X1 of exemple1 is
  signal a, b: std_logic;
  signal c   : std_logic_vector(3 downto 0);
begin
  --description de l'architecture
end architecture;
```

Notons que :

- On a créé 2 "**signal**" simple "a" et "b" et un bus "c" sur 4 bits.
- La ligne qui commence par -- est un **commentaire** en VHDL.

Règles d'écriture en VHDL

- Les instructions se terminent par ";". Elles peuvent s'écrire sur plusieurs lignes.
- Les noms des identificateurs (entité, architecture, signal, variable, labels...) ne doivent pas être des mots-clés du langage.

En ce qui concerne l'écriture de données, il faut respecter les règles suivantes :

- Un caractère simple est noté entre " ' " (simple quote). Exemples : 'd', '5', 'X'.
- Une chaîne de caractères est notée entre " " " (double quote). Exemples : "xyz", "A7".

Concernant les valeurs binaires :

- Un bit simple s'écrit entre " ' " : '0', '1'.
- Un vecteur s'écrit : en binaire "0101", "0011110" ou en notation hexadécimale X"A", qui correspond à "1010", X"B7" correspondant à "10110111". On peut aussi utiliser l'opérateur de concaténation : "01101101" = '0' & '1' & X"B" & "01"

Les principaux types utilisés

Le type boolean

Ce type est défini dans le package standard. C'est un type à 2 valeurs : (**false**, **true**). Il sert surtout à qualifier le résultat d'un test de comparaison.

Le type bit

Il est assez peu utilisé, car il se limite aux 2 valeurs '1' et '0'. Ce type répond à la théorie mathématique booléenne, mais ne suffit pas pour modéliser le comportement physique d'un circuit.

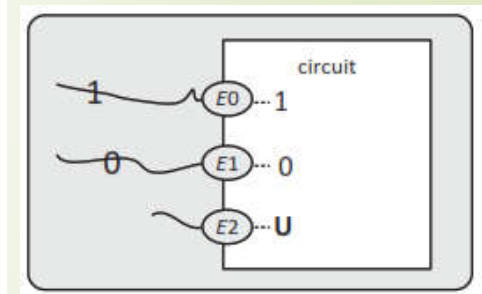
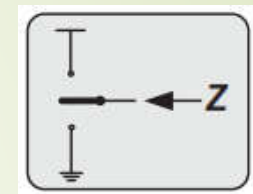
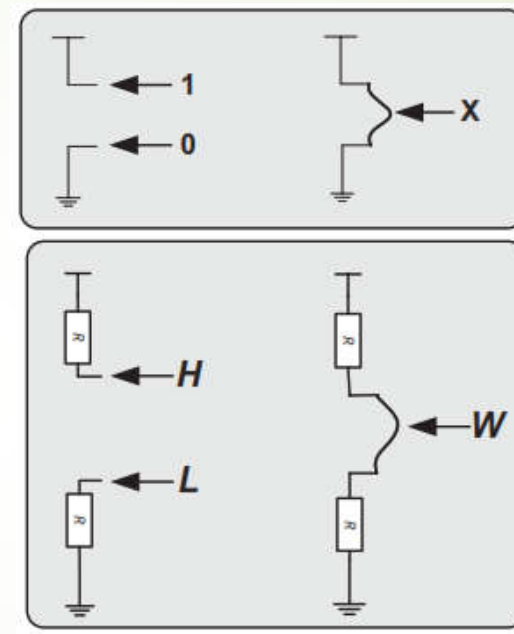
Le type std_logic

C'est le type fondamental du VHDL. Il est défini dans le package "std_logic_1164". Il modélise au mieux la réalité physique des circuits. Il définit ce qu'on appelle de la logique à **9 états**. C'est un type énuméré, c'est-à-dire que c'est une liste ordonnée finie de valeurs : ('U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-').

Les principaux types utilisés

Les 9 états du type `std_logic`:

- '0' niveau 0, forçage fort;
- '1' niveau 1, forçage fort;
- 'L' niveau 0, forçage faible;
- 'H' niveau 1, forçage faible;
- 'Z' haute impédance;
- 'X' niveau inconnu, forçage fort;
- 'W' niveau inconnu, forçage faible;
- '-' quelconque;
- 'U' non initialisé.



On peut également représenter une valeur sans importance par un tiret '-' qui se traduit par *don't care* en anglais. Mais en pratique, il est assez peu utilisé car cela laisse une incertitude sur le comportement du circuit concernant ces valeurs. L'outil de synthèse peut utiliser cet état pour optimiser les fonctions logiques, car il peut lui affecter indifféremment une valeur '1' ou '0'.

Les principaux types utilisés

On notera que pour les outils de synthèse, ces 9 états sont simplifiés. L'outil de synthèse adapte les 9 états comme noté dans le tableau suivant:

Valeur	Signification	Valeur en synthèse
0	niveau bas fort	0 logique
L	niveau bas faible	
1	niveau haut fort	1 logique
H	niveau haut faible	
Z	haute impédance	haute impédance
W	inconnu faible	non accepté
X	inconnu	sans importance
-	sans importance	
U	non initialisé	non accepté

Les principaux types utilisés

Les types `std_logic_vector` et `bit_vector`

On est très souvent amené à traiter des bus, c'est-à-dire un ensemble de valeurs en même temps. On appelle ceux-ci des vecteurs en VHDL. Un bus est simplement un ensemble de `std_logic` ou de `bits` que l'on peut traiter élément par élément ou globalement.

Prenons les exemples de vecteurs sur n bits du tableau suivant:

<code>std_logic_vector (n-1 downto 0)</code>	MSB de rang $n - 1$, LSB de rang 0
<code>std_logic_vector (0 to n-1)</code>	MSB de rang 0, LSB de rang $n - 1$
<code>std_logic_vector (2*n-1 downto n)</code>	MSB de rang $2 \times n - 1$, LSB de rang n

On utilise le `bit_vector` de la même manière que le `std_logic_vector`.

Les principaux types utilisés

Les types unsigned et signed

Ces types sont définis dans le package `numeric_std`. Ils ont été créés pour pouvoir faire des opérations sur des vecteurs. Ils donnent la valeur décimale entière signée ou non signée d'un `std_logic_vector`. Si c'est une valeur signée, la valeur binaire du vecteur est interprétée en code complément à 2.

Par exemple, Soit un signal `u: std_logic_vector (3 downto 0);`.
Affectons-lui une valeur binaire : `u <= "1010";` :

- La valeur non signée de `u` s'écrit `unsigned(u)` et vaut 10.
- Et la valeur signée de `u` s'écrit `signed(u)` et vaut -2.

Remarque : on peut affecter une valeur à un vecteur en le déclarant. Par exemple :

```
■ signal v: std_logic_vector (7 downto 0):="00001010";
```

Les principaux types utilisés

Autres types numériques

On trouve les types classiques **integer**, **natural** et **positive** du langage C.

On rappelle que **natural** correspond aux nombres entiers positifs ou nuls, et **positive** correspond aux nombres positifs non nuls. Ces 2 ensembles sont des sous types (**subtype**) du type **integer** en VHDL.

Exemple :

```
signal x: integer:=432;  
signal y: integer range 0 to 31:=12;
```

Les principaux types utilisés

Le type énuméré

C'est un type très utilisé dans les machines d'état. C'est en fait un type sur mesure que l'on crée de toute pièce en définissant une liste des différents éléments.

Par exemple:

```
■ type etat is (init, calcul, fin);
```

Ensuite, pour utiliser ce type "état", il faudra déclarer des signaux :

Le signal "futur" est déclaré comme étant du type "etat" et prend pour valeur initiale "init".

Les déclarations de signaux, de constantes et de tableaux

Signaux et constantes

Tous les objets doivent être définis en VHDL, et on peut adjoindre une valeur initiale en même temps que la déclaration.

On aura par exemple :

```
signal x: std_logic; --le signal x n'est pas initialise  
signal y: std_logic := '1'; --le signal y est initialise a '1'  
signal ad: std_logic_vector (3 downto 0) := x"F"; --ad est initialise a  
"1111"  
signal z: integer range 0 to 63 := 32; --l'entier peut varier de 0 a 63  
et vaut 32
```

Pour les constantes, il suffit de remplacer "signal" par "constant" et il faut lui affecter obligatoirement sa valeur :

```
constant vec: std_logic_vector (7 downto 0) := "001101110";--sur 8 bits
```


Les déclarations de signaux, de constantes et de tableaux

Création de tableaux

Il n'existe pas de tableau en VHDL. Il faut créer un type spécifique puis l'utiliser en créant un signal du type créé.

Par exemple :

```
type ROM is array (0 to 3) of std_logic_vector (7 downto 0); --type
tableau
constant my_mem: ROM := ( x"00",x"0F", x"1A", x"BB"); --constante
tableau
signal e: std_logic_vector (7 downto 0);
signal z: std_logic;
```

Pour lire une valeur du tableau *my_mem* de type ROM, on écrit :

```
e <= my_mem(2); -- e prend la valeur hexadecimale "1A"
```


VHDL concurrent et VHDL séquentiel

Il existe deux types de processus:

- ✓ le processus implicite ou **instruction concurrente**
- ✓ le processus explicite : ensemble d'instructions exécutées séquentiellement

Exemple:

```
architecture toto of test is
begin
  c <= a and b;
  z <= c when oe='1' else 'Z';
  seq: process (clk, reset)
  begin
    .
    .
    end process;
end toto;
```

} processus implicites

} processus explicite

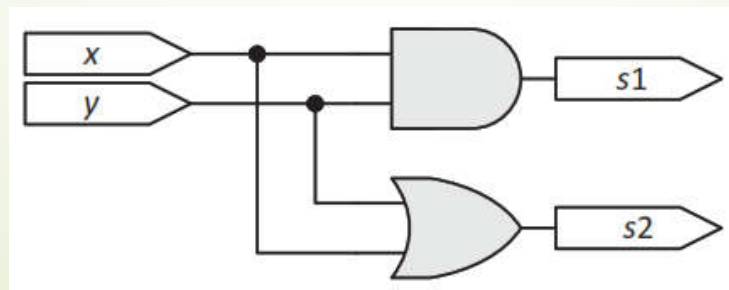
VHDL concurrent et VHDL séquentiel

Le VHDL concurrent

Le VHDL est un langage de description de matériel. Chaque instruction engendre de la logique. Prenons le code suivant :

```
architecture concurrent1 of exemple1 is
begin
  s1 <= x and y;
  s2 <= x or y;
end architecture;
```

On reconnaît la description d'une porte AND et d'une porte OR. Cette description engendre après synthèse le logigramme suivant:



VHDL concurrent et VHDL séquentiel

Le VHDL concurrent

Mais si on inverse l'ordre des instructions :

```
architecture concurrent1 of exemple1 is begin  
  s2 <= x and y;  
  s1 <= x or y;  
end architecture;
```

On obtient le même circuit. Ce qui signifie que l'ordre des instructions n'a aucune importance, car elles sont exécutées en même temps. C'est ce qu'on appelle du VHDL concurrent.

VHDL concurrent et VHDL séquentiel

Les principales instructions concurrentes

On distingue des instructions d'affectation, de manipulation de la hiérarchie et de génération de code.

Les opérations d'affectation se font exclusivement entre objets de même type et de même dimension.

Affectation simple

Elle permet essentiellement de recopier une valeur d'un signal à un autre.

Par exemple :

```
x <= y; -- recopie le signal y sur le signal x  
a <= b after 50 ns; -- affectation avec un délai
```

VHDL concurrent et VHDL séquentiel

Affectation conditionnelle

Elle traduit un multiplexeur. Par exemple :

```
■ x <= e0 when a='0' else e1;  --multiplexeur 2 vers 1
```

Voici également l'exemple d'un multiplexeur 4 vers 1. Soit le vecteur :

```
■ ad: std_logic_vector (1 downto 0);
```

On écrit :

```
■ s <= e0 when ad="00" else e1 when ad="01" else e2 when ad="10" else e3;
```

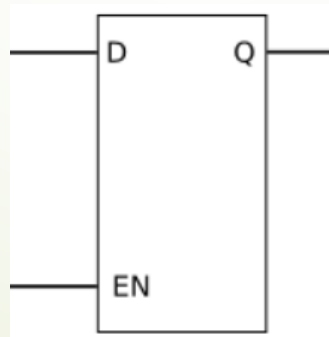
VHDL concurrent et VHDL séquentiel

En revanche, observons l'instruction suivante :

```
s <= e0 when a='1';
```

Elle fabrique un **latch** ayant pour horloge *a*. Car quand *a* = '0', **S** conserve la valeur de *e0*.

Et quand *a* = '1', **S** recopie *e0*.



VHDL concurrent et VHDL séquentiel

Affectation sélective

Cette instruction permet de traduire facilement la table de vérité d'une fonction. Par exemple, le multiplexeur 4 vers 1, il s'écrit :

```
with ad select
  when "00" => s <= e0,
  when "01" => s <= e1,
  when "10" => s <= e2,
  when "11" => s <= e3,
  when others => null;
```

La grammaire VHDL impose de tester toutes les combinaisons possibles de "ad". C'est pourquoi la dernière ligne est présente, qui englobe tous les cas non listés.

VHDL concurrent et VHDL séquentiel

Hiérarchie et instanciation

Il est très courant de réutiliser des circuits déjà réalisés. Ceci permet d'optimiser les études. On appelle ces circuits des **IP** (pour **I**ntellectual **P**roperty).

On procède en 2 temps : une première phase de déclaration de l'IP, puis la phase d'instanciation de l'IP dans le circuit. Bien sûr, pour que cela fonctionne, il faut que l'IP figure dans une bibliothèque du logiciel. La déclaration de l'IP se fait dans la zone de déclaration de l'architecture et son instanciation se fait dans le corps de l'architecture, comme on peut le voir sur l'extrait de code suivant.

```
architecture ...  
  -- déclaration de l'IP  
  component nom_entité_IP is [generic(--liste des generic);]  
  port (--liste des entrees / sorties);  
  end component;  
  [...]  
begin  
  -- instanciation de l'IP  
  label: nom_entité_IP [generic map (--cablage des generic)]  
    port map( --cablage des entrees / sorties);
```

VHDL concurrent et VHDL séquentiel

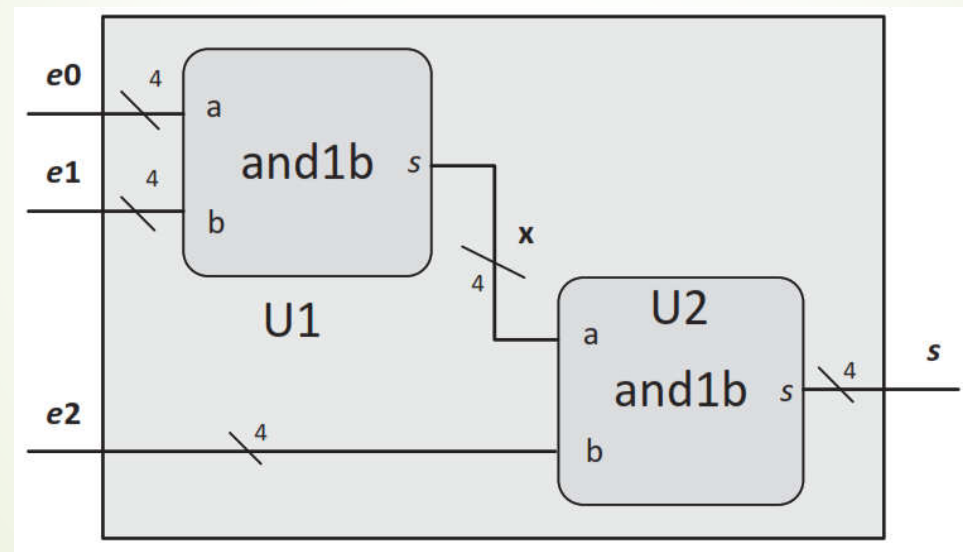
Par exemple, considérons le circuit suivant qui nous servira d'IP :

```
entity andlb is generic (n: natural:= 8);  
port (a,b: in std_logic_vector (n-1 downto 0);  
      s : out std_logic_vector (n-1 downto 0));  
end entity;  
architecture archi of andlb is  
begin  
  s <= a and not b;  
end architecture;
```

Ce circuit réalise un ET entre 2 vecteurs a et l'inverse de b de dimension paramétrée par le générique n.

VHDL concurrent et VHDL séquentiel

On utilise cet IP pour fabriquer le circuit "and1b_top" qui traite des bus sur 4 bits, dont le schéma structurel est décrit en Figure suivante:



Logigramme structurel du circuit

VHDL concurrent et VHDL séquentiel

Le code VHDL de ce circuit est:

```
entity and1b_top is port
  (e0, e1, e2: in std_logic_vector(3 downto 0);
   s      : out std_logic_vector(3 downto 0));
end entity ;
architecture hierarchique_87 of and1b_top is
  signal x : std_logic_vector(3 downto 0);
  -- declaration de l'IP
  component and1b is generic (n: natural:= 8);
  port (a, b: in std_logic_vector (n-1 downto 0);
        s  : out std_logic_vector (n-1 downto 0));
  end component;
begin
  -- cablages de l'IP
  U1: and1b generic map (n=>4) port map (a=>e0, b=>e1, s=>x);
  U2: and1b generic map (n=>4) port map (a=>x, b=>e2, s=>s);
end architecture ;
```

Notons que :

- La déclaration de l'IP est à l'image de son entité, et l'instanciation de 2 IP sur les labels U1 et U2. On a adapté la dimension des vecteurs à 4 bits en modifiant la valeur du **generic** lors du câblage.
- On remarque également le sens des flèches pour le câblage des IP. On relie les fils de l'intérieur de l'IP vers l'extérieur.
- De plus, il n'y a aucun lien entre le "s" déclaré dans l'IP et le "s" déclaré dans "and1b_top". Les noms des signaux ne traversent pas la hiérarchie.

Opérateurs en VHDL

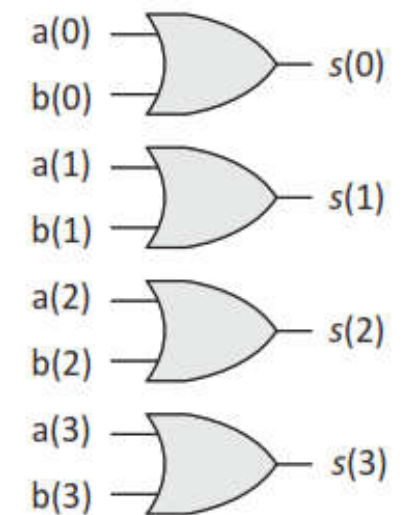
Opérateurs logiques

On trouve les opérateurs booléens classiques, que nous regroupons dans le Tableau suivant:

Opérateurs			
and	not	nand	xor
or		nor	xnor

Avec le VHDL, on ne peut utiliser ces opérateurs qu'entre signaux du même type (**bit** ou **std_logic**) et de même dimension pour les vecteurs.

Par exemple, si *a* et *b* sont des **std_logic_vector** (3 downto 0), l'instruction `s <= a or b;` engendre le circuit suivant:



Opérateurs en VHDL

Opérateurs relationnels

On peut classer, ordonner et comparer des nombres (**integer...**) ou des valeurs numériques de vecteurs (**signed** ou **unsigned**).

Symbole	Opération
>	supérieur
>=	supérieur ou égal
=	égal
<=	inférieur ou égal
<	inférieur
/=	différent

Opérateurs en VHDL

Opérateurs arithmétiques

Ces opérations ne concernent que les nombres et les valeurs numériques de vecteurs.

Symbole	Opération
+	addition
-	soustraction
*	multiplication
/	division
**	élévation à la puissance
mod	modulo
rem	reste

Opérateurs en VHDL

Les opérateurs de décalage

Il existe des opérateurs prédéfinis pour matérialiser les décalages dans un vecteur. Avec ces opérateurs, il faut préciser le nombre de bits du décalage.

Soit un vecteur a : `std_logic_vector (7 downto 0):= "01010011"`; et x un vecteur de même dimension.

Mot clé	Signification en anglais	Syntaxe	Valeur de x
sll	<i>Shift left logical</i>	x <= a sll 2;	"01001100"
srl	<i>Shift right logical</i>	x <= a srl 3;	"00001010"
sla	<i>Shift left arithmetic</i>	x <= a sla 2;	"01001111"
sra	<i>Shift right arithmetic</i>	x <= a sra 3;	"00001010"
rol	<i>Rotate left</i>	x <= a rol 5;	"01101010"
ror	<i>Rotate right</i>	x <= a ror 5;	"10011010"

Opérateurs en VHDL

Les opérateurs de décalage

On utilise plus souvent l'opérateur de concaténation "&" pour matérialiser les décalages, car il est plus explicite. On a traduit dans le Tableau suivant l'équivalent des opérateurs de décalage.

Mot clé	Syntaxe	Syntaxe avec l'opérateur de concaténation
sll	<code>x <= a sll 2;</code>	<code>x <= a(5 downto 0) & "00";</code>
srl	<code>x <= a srl 3;</code>	<code>x <= "000" & a(7 downto 3);</code>
sla	<code>x <= a sla 2;</code>	<code>x <= a(5 downto 0) & a(0) & a(0);</code>
sra	<code>x <= a sra 3;</code>	<code>x <= a(7) & a(7) & a(7) & a(7 downto 5);</code>
rol	<code>x <= a rol 5;</code>	<code>x <= a(2 downto 0) & a(7 downto 3);</code>
ror	<code>x <= a ror 5;</code>	<code>x <= a(4 downto 0) & a(7 downto 5);</code>

Opérateurs en VHDL

Autres éléments de VHDL

Très souvent, on est amené à écrire globalement une valeur sur un vecteur, en particulier lors d'une initialisation. On écrit alors :

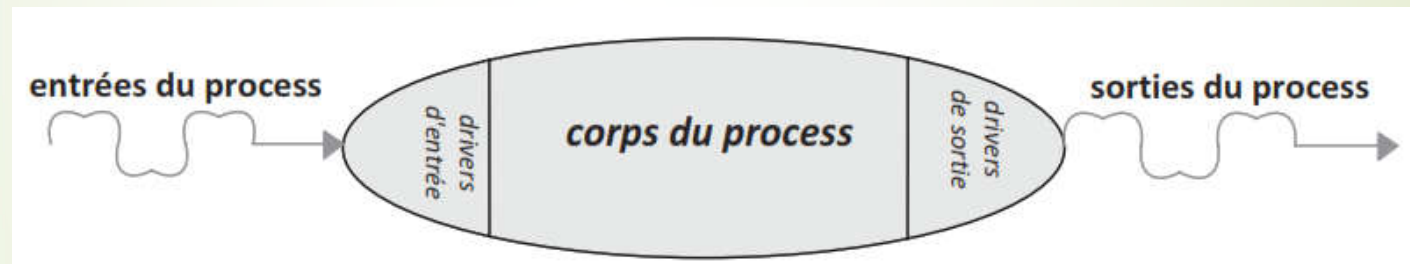
```
x <= (others => '0'); -- tout le vecteur est mis à 0  
y <= (others => '1'); -- tout le vecteur est mis à 1
```

Cette instruction est valable quelle que soit la taille des vecteurs x et y.

VHDL séquentiel

Introduction:

C'est la manière d'écrire la plus utilisée, car elle permet de décrire le **comportement** d'un circuit au plus haut niveau. Il est décrit avec des instructions séquentielles. Pour cela, on utilise ce qu'on appelle un process en VHDL.



Modélisation d'un process

Le process est constitué de drivers d'entrée qui permettront de stocker les valeurs d'entrées pendant toute la durée de l'exécution du process, et des drivers de sorties qui stockeront les sorties tant que le process n'a pas terminé son exécution. Le corps du process exécute les instructions séquentiellement.

VHDL séquentiel

Description d'un process

Un process commence par le mot clé "**process**" et se termine par le mot clé "**end process**". Après le mot clé "**process**" et le nom du process, on trouve éventuellement une liste de signaux entre parenthèses. Cette liste s'appelle la liste de sensibilité du process. En simulation, on utilise une autre manière d'écrire le process sans liste de sensibilité. On trouve les instructions séquentielles après le mot clé "**begin**" jusqu'à la fin du process matérialisée par "**end process**".

Process à liste de sensibilité

Cette liste sert à déclencher le déroulement de l'algorithme décrit dans le process. S'il y a un changement d'une des valeurs de la liste de sensibilité, le calcul est relancé après avoir terminé. On dit que le process est réévalué. Sinon, il est inutile de relancer le calcul.

VHDL séquentiel

Par exemple, le process suivant sera réévalué à chaque changement de valeur de a ou de b.

```
process(a, b) is  
begin  
  s <= a and b;  
end process;
```

Cette liste est très importante en simulation. Le process est exécuté par le simulateur à chaque changement d'une des valeurs de cette liste.

VHDL séquentiel

Process combinatoire et process séquentiel

On distingue 2 types de **process** :

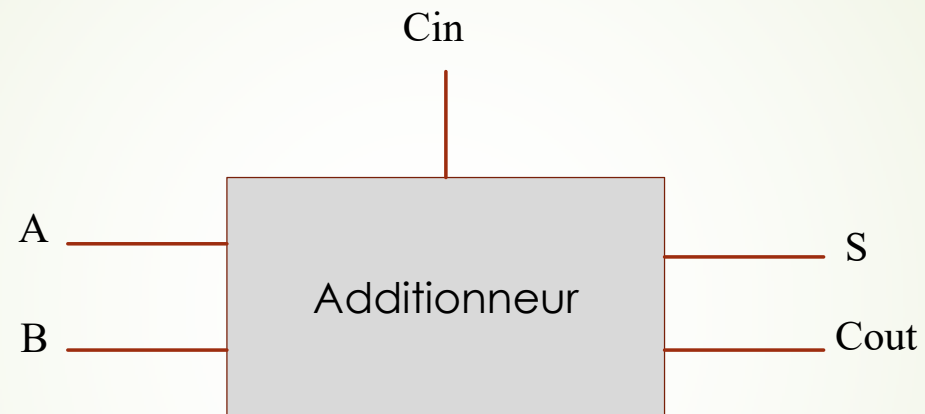
- **Process synchrones** qui décrivent des registres et
- **Process combinatoires** qui décrivent de la logique combinatoire.

Dans un **process synchrone**, la **liste de sensibilité** se limite à l'horloge et à l'entrée d'initialisation asynchrone du registre. En effet, la sortie du registre ne change qu'au front montant d'horloge ou lors de l'initialisation asynchrone du registre. Il est donc inutile de calculer les valeurs des sorties en dehors de ces instants.

Dans un **process combinatoire**, la **liste de sensibilité** doit comporter tous les signaux qui interviennent sur les sorties du process. En effet, il faut réévaluer les sorties à chaque changement d'une des entrées du process.

Exercice d'application

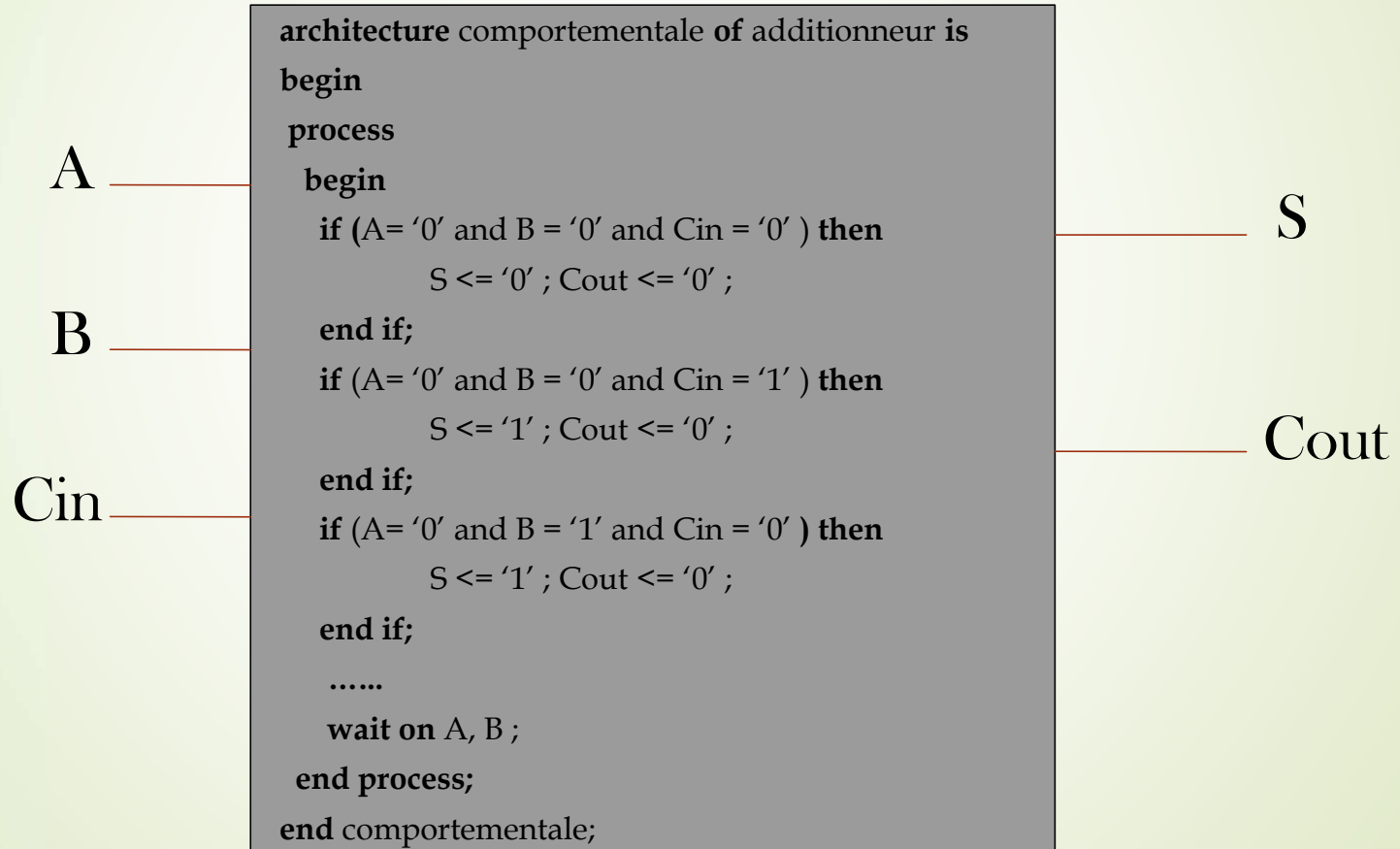
Additionneur complet



Donner le modèle comportementale, structurel et data flow de l'additionneur complet.

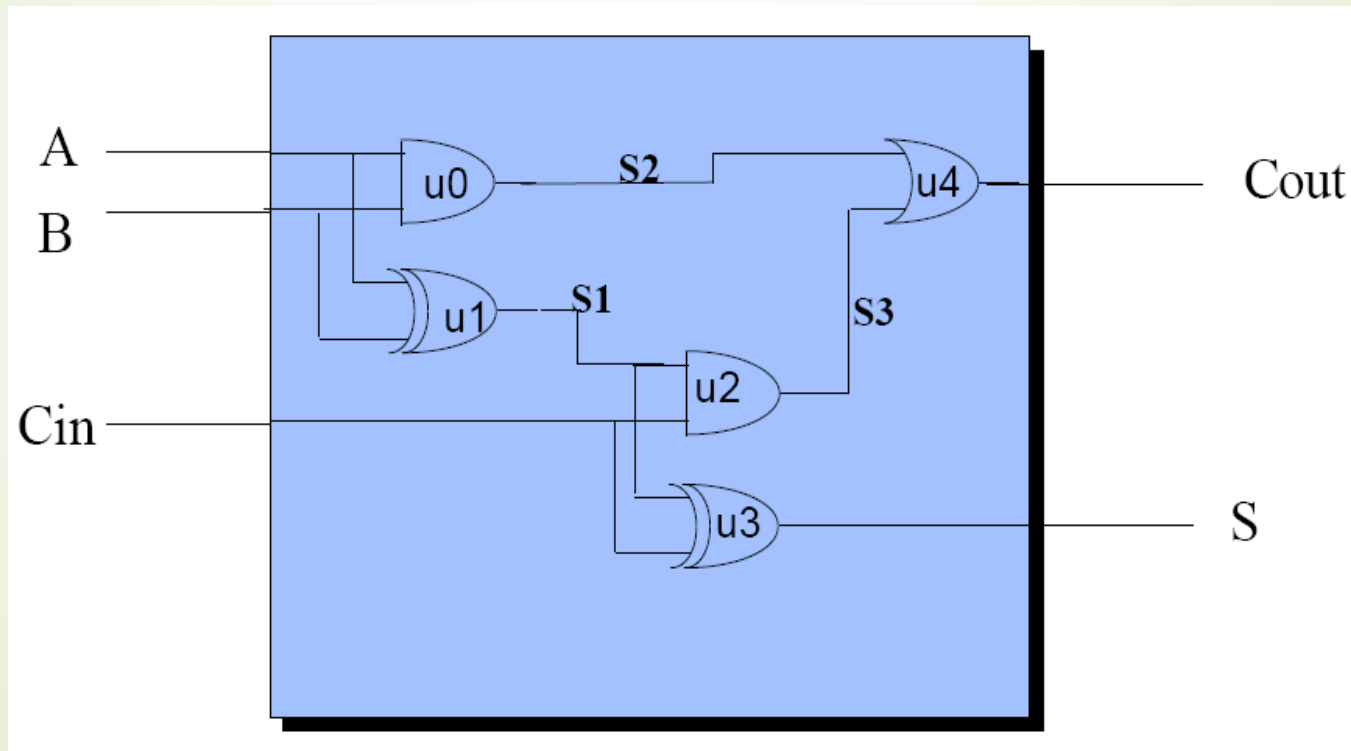
Exercice d'application

Modèle comportemental:



Exercice d'application

Modèle structurel:



Exercice d'application

Modèle structurel:

```
architecture structurelle of additionneur is

  component porte_OU port ( e1 : in bit;
                           e2 : in bit;
                           s  : out bit );
  end component ;

  component porte_ET port ( e1 : in bit;
                           e2 : in bit;
                           s  : out bit );
  end component ;

  component porte_XOR port ( e1 : in bit;
                             e2 : in bit;
                             s  : out bit );
  end component ;
```

```
Signal s1, s2, s3 : bit ;

begin

  u0: porte_ET port map (A, B, S2) ;

  u1: porte_XOR port map (A, B, S1) ;

  u2: porte_ET port map (S1, Cin, S3) ;

  u3: porte_XOR port map (S1, Cin, S) ;

  u4: porte_OU port map (S2, S3, Cout) ;

end structurelle ;
```

Exercice d'application

Modèle Data flow:

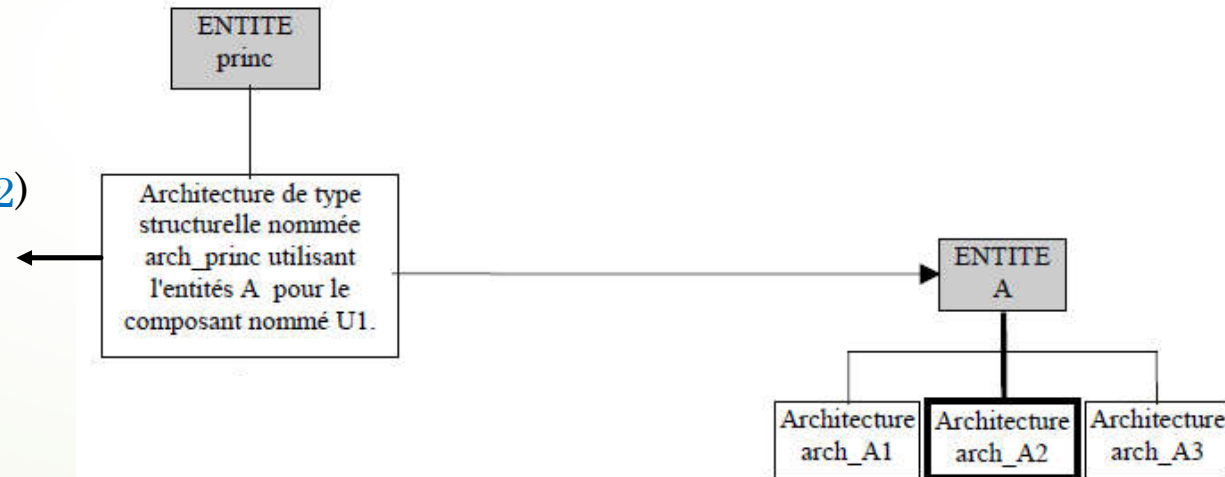
```
architecture data_flow of additionneur is
begin
    S <= A xor B xor Cin;
    Cout <= A and B or (A xor B) and Cin
end data_flow;
```

Unités de conception

Configuration

Si l'entité **A** est utilisée au sein de l'architecture *arch_princ* de l'entité *princ*, et si on a plusieurs architectures pour cette entité **A**. Lors de la synthèse ou de la simulation de l'entité *princ*, il va être nécessaire de spécifier quelles sont les architectures à utiliser. Cette spécification peut être faite grâce à l'utilisation d'une configuration.

```
Configuration conf_princ of Princ is
  for arch_princ
    for U1 : A use entity work.A(arch_A2)
    end for;
  end for;
end conf_princ;
```



Unités de conception

Configuration

- La configuration permet, comme son nom l'indique, de configurer l'entité à laquelle elle est associée.
- Pour qu'une description soit portable, c'est-à-dire synthétisable ou simulable sur des outils différents et pour des composants cibles différents, il est préférable de n'avoir qu'une seule architecture par entité.

Unités de conception

Package:

Dans le langage VHDL, il est possible de créer des “package” et “package body” dont les rôles sont de permettre le **regroupement** de **données, variables, fonctions, procédures**, etc., que l'on souhaite pouvoir utiliser ou appeler à partir des architectures.

Unités de conception

Package:

- ✓ Décrit une vue externe (boite noire)
- ✓ Regroupement de déclaration de type et/ou sous programme.
- ✓ Construction d'une bibliothèque.
- ✓ Le contenu de la déclaration du paquetage est visible de l'extérieur.
- ✓ Déclaré avant l'entité.

Package Body:

- ✓ Décrit une vue interne (comment de la boite noire)
- ✓ Contient l'écriture proprement dites des fonctions et des procédures déclarées au niveau du paquetage.
- ✓ Le corps du package n'est pas toujours nécessaire.

Unités de conception

Package:

```
package PACK is
-- déclarations de types, sous types, signaux, composants
-- constantes, sous programmes (sans code)
-- aucune variable
end PACK;
```

Package Body:

```
Package body PACK is
-- déclarations identiques (sauf signaux)
-- corps des SP de la partie déclarative
end PACK;
```

Unités de conception

Exemple:

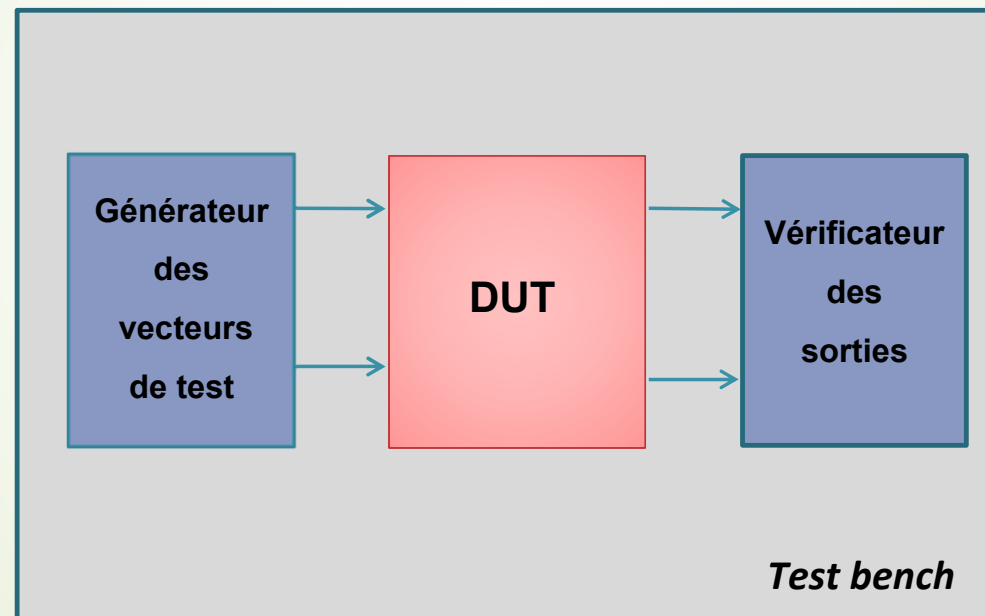
```
Package geometrie is  
  
constant pi :real := 3.141592654;  
procedure aire_cercle ( a: in real ; b: out real);  
  
end geometrie;  
  
package body geometrie is  
  
procedure aire_cercle ( a: in real ; b: out real) is  
    begin  
        b := pi * a**2 ;  
    end;  
end geometrie;
```

```
use work.geometrie.pi;  
use work.geometrie.aire_cercle;  
.....
```

```
calc: process(h)  
    variable aire, perim :real ;  
    begin  
        ....  
        aire_cercle (rayon,aire);  
        perim := 2*pi*rayon;  
        ....  
    end process calc;  
.....
```

Simulation par Test bench

Un **testbench** (banc de test) est une description VHDL incorporant une instance du circuit à tester (**DUT**) et lui fournissant les séquences de signaux d'entrées (stimuli) afin de visualiser les sorties avec un simulateur.



NB: Cas particulier de la simulation : circuit "top" sans entrée ni sortie

Simulation par Test bench

```
Library IEEE ;
Library work ;
use ieee.std_logic_1164.all ;
use work.all ;

entity bascule_d_tb is
end ;

architecture bascule_d_tb of bascule_d_tb is

    signal clk, reset, d, q : std_logic ;

    component bascule_d port ( reset, clk, d : in std_logic ;
                               q : out std_logic );

    end component ;
```

Simulation par Test bench

```
begin

DUT : bascule_d  port map ( clk => clk , reset => reset , d => d , q => q ) ;

clk_in : process
    Begin
        clk <='0'; wait for 20 ns ;
        clk <='1'; wait for 20 ns ;
    end process clk_in ;

Reset <= '1' , '0' after 10 ns ;

d_in : process
    begin
        d <='1'; wait for 30 ns;
        d <='0'; wait for 60 ns;
        d <='1'; wait for 90 ns;
    end process d_in;
end ;
```